

บรรยาย 4

swap

```
def swap(a, i, j):
    tmp = a[i]
    a[i] = a[j]
    a[j] = tmp
    return a
```

บรรยาย 4

Bubble Sort

~ N*N

```
def bubbleSort(a, N):
    for i in range(N):
        for j in range(N - 1, 0, -1):
            if a[j] < a[j - 1]:
                a = swap(a, j, j - 1)
        return a

data = [8, 7, 2, 1]
data_sorted = bubbleSort(data, len(data))
print(data_sorted)
[1, 2, 7, 8]
```

ลูบใน j ึ่งถอยหลัง N-1 → 1 ึ่งดันค่าน้อยขึ้นหน้า

บรรยาย 4

Selection Sort

~ N*N

```
def selectionSort(a, N):
    for i in range(0, N - 1):
        min = i
        for j in range(i + 1, N):
            if a[j] < a[min]:
                min = j
        swap(a, min, i)
    return a

key = [8, 7, 2, 1]
key_sorted = selectionSort(key, len(key))
[1, 2, 7, 8]
```

หาตำแหน่งค่าน้อยสุดเก็บใน min สลับครั้งเดียวตอนจบ

บรรยาย 4

Insertion Sort

ดีสุด N

```
def insertionSort(a, n):
    for i in range(1, n):
        key = a[i]
        j = i - 1
        while j >= 0 and a[j] > key:
            a[j + 1] = a[j]
            j -= 1
        # Insert key
        a[j + 1] = key

key = [8, 7, 2, 1]
insertionSort(key, len(key))
print(key)
[1, 2, 7, 8]
```

เหมือนเรียงไฟในมือ · ไม่มี return แก่ list เติมโดยตรง

บรรยาย 4

เปรียบเทียบความเร็ว

วิธี	ความเร็ว	หมายเหตุ
Bubble	~ N*N	ง่ายแต่ช้า
Selection	~ N*N	ประมาณ Bubble
Insertion	N*N	เรียงมาดีแล้ว = N

วิเคราะห์จากจำนวนรอบของคำสั่งที่ถูกเรียกมากที่สุด คือ ลูบซ้อนสองชั้น

บรรยาย 5

Quick Sort

```
Quick-Sort(A, left, right)
    if left >= right
        return
    else
        middle ← Partition(A, left, right)
        Quick-Sort(A, left, middle - 1)
        Quick-Sort(A, middle + 1, right)
    end if

เลือก pivot p เช่นตัวแรก · ส่วนแรกคือที่ < p ส่วนที่สองคือ ≥ p
```

บรรยาย 4

เรียง record ด้วย

กฎแฉกเป็น column

```
def insertionSortRec(r, n, key_col):
    for i in range(1, n):
        key_rec = r[i]
        j = i - 1
        while j >= 0 and \
            r[j][key_col] > key_rec[key_col]:
            r[j + 1] = r[j]
            j -= 1
        # Insert key
        r[j + 1] = key_rec
```

เทียบที่คอลัมน์ [key_col] แต่ย้ายทั้งแถว

บรรยาย 4

ตัวอย่างข้อมูล record

```
data = [
    ['Mark', "123-65-6789", 75, 78, 82], \
    ['Tim', "123-65-5723", 67, 45, 74], \
    ['Amy', "123-65-4542", 78, 65, 90], \
    ['Berk', "123-65-8278", 81, 74, 68] ]
key_column = 4
insertionSortRec(data, len(data),
    key_column)

for row in data:
    print(row)
['Berk', ..., 81, 74, 68]
['Tim', ..., 67, 45, 74]
['Mark', ..., 75, 78, 82]
['Amy', ..., 78, 65, 90]
```

ถ้าคิดเฉพาะกฎแฉกมาเรียงจะได้แค่ [68, 74, 82, 90] แถวไม่ย้ายตาม

บรรยาย 4

อ่าน record จากไฟล์ CSV

```
import csv

def readRecordToList(filename):
    infile = open(filename, 'r')
    mylist = []
    heading = next(infile)
    csv_obj = csv.reader(infile)
    for row in csv_obj:
        mylist.append(row)
    infile.close()
    return mylist

next(infile) ข้ามบรรทัดหัวตาราง · ได้ List ซ้อน List
```

กั้บดั้ก บรรยาย 4

- Bubble ลูบในจบที่ 1 ไม่ใช่ 0 เพราะต้องอ้าง a[j-1]
- insertionSort และ insertionSortRec ไม่มี return
- ทุกฟังก์ชันรับ N ต้องส่ง len(a) ไปด้วยเสมอ

บรรยาย 5

Divide and Conquer

- 1. Base Case แก่ตรง ๆ ถ้าเล็กพอ
- 2. Divide แบ่งเป็นปัญหาย่อยที่เล็กลง
- 3. Recursively solve เรียกตัวเองแก้ปัญหาย่อย
- 4. Combine รวมคำตอบของปัญหาย่อย

MERGE-SORT A[0..N-1], Left, Right

- If N = 1, done.
- m = N/2
- Merge-Sort(A, Left, m)
- Merge-Sort(A, m + 1, Right)
- "Merge" the 2 sorted lists.

บรรยาย 5

Merge vs Quick

	MERGE	QUICK
งานหลัก	ตอน merge	ตอน partition
Worst	O(n log n)	O(n²)
หน่วยความจำ	More	Less (in-place)
Stability	Stable	Not stable

Stable = ตัวที่ค่าเท่ากันยังคงลำดับเดิม · ทั้งคู่ดีกว่า Insertion Sort

บรรยาย 5

ฟังก์ชันเรียกตัวเอง

ลอกสอบนั่น

```
def message(times):
    print("message is called, times = ",
        times)
    if times > 0:
        message(times - 1)
    # -----
    print("message return with ", times)

if __name__ == '__main__':
    message(3)
message is called, times = 3
message is called, times = 2
message is called, times = 1
message is called, times = 0
message return with 0
message return with 1
message return with 2
message return with 3
```

ขาไปนับ 3→0 · ขากลับนับ 0→3 · message(3) เรียกตัวเอง 4 ครั้ง

บรรยาย 5

merge

O(N)

```
def merge(A, p, q, r):
    # slicing creates a copy
    if type(A) is list:
        left = A[p:q+1]
        right = A[q+1:r+1]
    else:
        left = list(A[p:q+1])
        right = list(A[q+1:r+1])
    i = 0 # index into left
    j = 0 # index into right
    k = p # index into A[p:r+1]

    while i < len(left) and j < len(right):
        if left[i] <= right[j]:
            A[k] = left[i]
            i += 1
        else:
            A[k] = right[j]
            j += 1
        k += 1
    if i < len(left):
        A[k:r+1] = left[i:]
    if j < len(right):
        A[k:r+1] = right[j:]

A = [2, 3, 7, 8, 5, 6, 8, 9]
merge(A, 0, 3, 7)
[2, 3, 5, 6, 7, 8, 8, 9]
```

เขียนผลลง A ตัวเดิม ไม่คืนค่าใหม่ · ครั้งแรก p..q ครั้งหลัง q+1..r

บรรยาย 5

merge_sort

N LOG N

```
def merge_sort(A, p=0, r=None):
    if r is None:
        r = len(A) - 1
    if p >= r: # 0 or 1 element
        return
    q = (p + r) // 2 # midpoint
    merge_sort(A, p, q)
    merge_sort(A, q + 1, r)
    merge(A, p, q, r)

A = [8, 3, 2, 9, 7, 1, 5, 4]
merge_sort(A)
print(A)
[1, 2, 3, 4, 5, 7, 8, 9]
```

กั้บดั้ก บรรยาย 5

- merge_sort ไม่ return และเรียกครั้งแรกแค่ merge_sort(A) เพราะ p, r มีค่าตั้งต้น
- ครั้งหลังเริ่มที่ q + 1 ไม่ใช่ q
- print ก่อนเรียกตัวเอง = ขาไป · หลังเรียกตัวเอง = ขากลับ

บรรยาย 6คลาสและวัตถุ

```
class birds():
    name = 'eagle'
    def fly(self):
        if self.name == 'eagle':
            print("I am an ", self.name,
                  " and I can fly")

eagle = birds()
eagle.fly()           # วิธีที่ 1
birds.fly(eagle)      # วิธีที่ 2
```

self ต้องเป็นพารามิเตอร์ตัวแรกเสมอ และอ่านตัวแปรในคลาสต้องใช้ self เสมอ (เหมือน this ใน Java) เรียกเมทอดได้ 2 วิธีตามด้านบน

ADT = properties + operations · class = ตัวแปรของวัตถุ + เมทอด ผู้ใช้ไม่ต้องรู้รายละเอียดภายใน

บรรยาย 6Constructor และ __str__

```
class rectangle():
    def __init__(self, x, y):
        self._x = x
        self._y = y
    def calArea(self):
        print("Area is ",
              self._x * self._y)
    def __str__(self):
        s = "Width: " + str(self._x) \
            + "\nHeight: " + str(self._y)
        return s

r1 = rectangle(2.0, 3.0)
print(r1)
r1.calArea()
Width: 2.0
Height: 3.0
Area is 6.0
```

`__init__` ถูกเรียกครั้งเดียวตอนสร้างวัตถุ · ตัวแปรปกปิดขึ้นต้นด้วย `_` · `__str__` ถูกเรียกเมื่อ `print`

บรรยาย 6คลาส Stack

FIFO

```
class Stack:
    def __init__(self, size):
        self._top = -1
        self._data = [None] * size
        self._size = size

    def isFull(self):
        if (self._top + 1) == self._size:
            return True
        else:
            return False

    def push(self, item):
        if self.isFull() == True:
            print("Stack is full : ", item)
        else:
            self._top += 1
            self._data[self._top] = item
            print("Pushed element: ", item)

    def isEmpty(self):
        if self._top == -1:
            return True
        else:
            return False

    def pop(self):
        if (self.isEmpty() == False):
            item = self._data[self._top]
            self._top -= 1
            return item
        else:
            print("Stack is empty")
            return -1
```

อย่าลืม push พิมพ์ "Pushed element: " ทุกครั้งที่ใส่สำเร็จ

บรรยาย 6ผลการรับ Stack

ออกสอบ

```
s1 = Stack(3)
s1.push(5)
s1.push(6)
s1.push(1)
s1.push(4)
d = s1.pop(); print("popped: ", d)
d = s1.pop(); print("popped: ", d)
d = s1.pop(); print("popped: ", d)
d = s1.pop()

Pushed element: 5
Pushed element: 6
Pushed element: 1
Stack is full : 4
popped: 1
popped: 6
popped: 5
Stack is empty
```

Stack ขนาด 3 · ค่า 4 ถูกปฏิเสธ · pop ได้ 1 → 6 → 5 แล้วครั้งที่ 4 ว่างจึงคืน -1

บรรยาย 6การประยุกต์ใช้ Stack

- จดจำย้อนกลับ Backtracking เช่น undo
- ใน compiler ส่งค่าไปยังโปรแกรมย่อย
- คำนวณพจน์คณิตศาสตร์ (1 + 2) * 3
- เรียงอักษรย้อนกลับ **THAI → IAHT**

คำนวณพจน์ใช้ Stack 2 อัน — Operand Stack เก็บตัวเลข, Operator Stack เก็บเครื่องหมาย · เจอตัวเลข/เครื่องหมาย push ลงกองของมัน · เจอ (ไม่ทำอะไร · เจอ) pop ตัวเลข 2 ตัวและเครื่องหมาย 1 ตัว คำนวณแล้ว push ผลกลับ · ((4 + 2) * 3) ได้ **18**

บรรยาย 7คลาส Node

```
class Node():
    def __init__(self, datum):
        self.data = datum
        self.next = None
    def getData(self):
        return self.data
    def __str__(self):
        return str(self.data)
```

โหนด = ข้อมูล + ตัวชี้ next ตัวท้ายชี้ None · data กับ next ไม่มีขีดล่างนำหน้า คือเป็น public

บรรยาย 7LinkedList · การเพิ่มโหนด

```
class LinkedList():
    def __init__(self):
        self._head = None
        self._tail = None
        self._size = 0

    def insertAtTail(self, datum):
        new_node = Node(datum)
        if self._tail == None:
            self._tail = new_node
            self._head = new_node
        else:
            self._tail.next = new_node
            self._tail = self._tail.next

    def insertAtHead(self, datum):
        new_node = Node(datum)
        if self._head == None:
            self._tail = new_node
            self._head = new_node
        else:
            new_node.next = self._head
            self._head = new_node
```

มี `_tail` เก็บไว้ `insertAtTail` จึงไม่ต้องไล่ทั้งรายการ

บรรยาย 7removeAtHead และ __str__

```
def removeAtHead(self):
    if (self._head != None):
        datum = self._head.data
        self._head = self._head.next
        return datum
    else:
        print("List is empty")
        return None

def __str__(self):
    curr = self._head
    i = 1
    s = ""
    while curr != None:
        s = s + "node : " + str(i) \
            + " " + str(curr) + "\n"
        curr = curr.next
        i += 1
    return(s)

mylist = LinkedList()
mylist.insertAtTail(["65-1", "Mark"])
mylist.insertAtTail(["65-2", "Ed"])
mylist.insertAtTail(["65-3", "Ty"])
print(mylist)
node : 1 ['65-1', 'Mark']
node : 2 ['65-2', 'Ed']
node : 3 ['65-3', 'Ty']
```

บรรยาย 7Queue

FIFO

```
from collections import deque

class Queue:
    def __init__(self):
        self.items = deque()
    def is_empty(self):
        return len(self.items) == 0
    def enqueue(self, item):
        self.items.append(item)
    def dequeue(self):
        if self.is_empty():
            raise IndexError(
                "Queue is empty")
        return self.items.popleft()
    def size(self):
        return len(self.items)
```

ใส่ด้านหลัง rear ด้วย `append` · เอาออกด้านหน้า front ด้วย `popleft` · **สไลด์เรียก is_empty()** แต่ไม่ได้เขียนเมทอดนี้ไว้ ถ้าลอกตามจะ error ต้องเขียนเพิ่มเอง

บรรยาย 7Circular และ Doubly

Circular เชื่อมเป็นวงกลม จำหัวท้าย หรือตำแหน่งหนึ่งในวง · **Doubly** มีทั้งแปล่า dummy จำหัวและท้าย มีตัวชี้ prev และ next

```
# ใส่ก่อนใหม่ด้านหน้า Doubly
1. new_node.prev = header
2. new_node.next = header.next
3. header.next.prev = new_node
4. header.next = new_node
```

ก้นดัก บรรยาย 6-7

- `insertAtHead` ต้อง `new_node.next = self._head` ก่อน แล้วจึง `self._head = new_node`
- LinkedList ใช้ `_head` `_tail` มีขีดล่าง แต่ Node ใช้ `data` `next` ไม่มีขีดล่าง
- ใส่รายการใช้ `curr` เดิน ห้ามเลื่อน `self._head`